```
# 05_evaluate.py
# Turns the saved predictions into the numbers and figures for the slides:
# per-label thresholds (tuned on VALIDATION only), test metrics, NDVI-vs-RGB significance,
# precision/recall curves, confusion matrices. No GPU needed. Run after 04.
```

```
# %% [1] Imports and settings
import os, json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import (fbeta_score, hamming_loss, recall_score,
                             precision_recall_curve, multilabel_confusion_matrix)

ROOT = os.environ.get("AMAZON_ROOT", "/content/drive/MyDrive/amazon_project")
try:
    from google.colab import drive
    drive.mount("/content/drive")
except ImportError:
    pass

CACHE, RUNS, FIG = f"{ROOT}/cache", f"{ROOT}/runs", f"{ROOT}/figs"
os.makedirs(FIG, exist_ok=True)
LABELS = json.load(open(f"{CACHE}/labels.json"))
NAMES = ["A_scratch_rgb", "B_scratch_nir_ndvi", "C_resnet_rgb", "D_resnet_nir_ndvi"]
NAMES = [n for n in NAMES if os.path.exists(f"{RUNS}/{n}_probs.npz")]
RARE = [l for l in ["artisinal_mine", "artisanal_mine", "conventional_mine",
                    "selective_logging", "slash_burn"] if l in LABELS]
RARE_IDX = [LABELS.index(l) for l in RARE]
print("Experiments found:", NAMES, "| rare labels:", RARE)
```

```
# %% [2] Helpers
def best_thresholds(y, p, grid=np.arange(0.02, 0.81, 0.01)):
    """Per-label threshold that maximises F2 on the validation set."""
    thr = np.zeros(y.shape[1])
    for j in range(y.shape[1]):
        pred = p[:, j:j + 1] >= grid[None, :]
        pos = y[:, j:j + 1] == 1
        tp = (pred & pos).sum(0)
        fp = (pred & ~pos).sum(0)
        fn = (~pred & pos).sum(0)
        thr[j] = grid[(5 * tp / (5 * tp + 4 * fn + fp + 1e-9)).argmax()]
    return thr


def f2_per_label(y, pred):
    tp = (pred & (y == 1)).sum(0)
    fp = (pred & (y == 0)).sum(0)
    fn = (~pred & (y == 1)).sum(0)
    return 5 * tp / (5 * tp + 4 * fn + fp + 1e-9)


def metrics(y, pred):
    rec = recall_score(y, pred.astype(int), average=None, zero_division=0)
    return {
        "macro_F2": fbeta_score(y, pred, beta=2, average="macro", zero_division=0),
        "samples_F2 (Kaggle metric)": fbeta_score(y, pred, beta=2, average="samples",
                                                  zero_division=0),
        "micro_F2": fbeta_score(y, pred, beta=2, average="micro", zero_division=0),
        "hamming_loss": hamming_loss(y, pred),
        "rare_recall_mean": float(rec[RARE_IDX].mean()),
        **{f"recall_{l}": float(rec[LABELS.index(l)]) for l in RARE},
    }
```

```
# %% [3] Thresholds from validation, metrics on test
res, store = {}, {}
for n in NAMES:
    d = np.load(f"{RUNS}/{n}_probs.npz")
    thr = best_thresholds(d["val_y"], d["val_probs"])
    pred = d["test_probs"] >= thr[None, :]
    res[n] = metrics(d["test_y"], pred)
    store[n] = dict(y=d["test_y"], pred=pred, probs=d["test_probs"], thr=thr)
    json.dump(dict(zip(LABELS, thr.tolist())), open(f"{RUNS}/thresholds_{n}.json", "w"))
table = pd.DataFrame(res).T
table.to_csv(f"{FIG}/results_table.csv")
print(table.round(4).to_string())
```

```
# %% [4] Table to paste into slide 14
rows = ["| Experiment | Macro F2 | Rare-class recall | Hamming loss | Kaggle-style F2 |",
        "|---|---|---|---|---|"]
for n in NAMES:
    r = res[n]
    rows.append(f"| {n} | {r['macro_F2']:.3f} | {r['rare_recall_mean']:.3f} | "
                f"{r['hamming_loss']:.4f} | {r['samples_F2 (Kaggle metric)']:.3f} |")
open(f"{FIG}/slide14_table.md", "w").write("\n".join(rows))
print("\n".join(rows))
```

```
# %% [5] Is NDVI really better? Paired bootstrap on the test set (95% interval)
rng = np.random.RandomState(0)


def boot_diff(a, b, reps=1000):
    y, pa, pb = store[a]["y"], store[a]["pred"], store[b]["pred"]
    diffs = []
    for _ in range(reps):
        i = rng.randint(0, len(y), len(y))
        diffs.append(f2_per_label(y[i], pb[i]).mean() - f2_per_label(y[i], pa[i]).mean())
    return np.mean(diffs), np.percentile(diffs, 2.5), np.percentile(diffs, 97.5)


pairs = [("A_scratch_rgb", "B_scratch_nir_ndvi", "NDVI effect, scratch CNN (B - A)"),
         ("C_resnet_rgb", "D_resnet_nir_ndvi", "NDVI effect, ResNet (D - C)"),
         ("A_scratch_rgb", "C_resnet_rgb", "Transfer learning, RGB (C - A)"),
         ("B_scratch_nir_ndvi", "D_resnet_nir_ndvi", "Transfer learning, +NDVI (D - B)")]
boot_rows = []
for a, b, label in pairs:
    if a in store and b in store:
        m, lo, hi = boot_diff(a, b)
        verdict = "clear gain" if lo > 0 else ("clear loss" if hi < 0 else "not significant")
        boot_rows.append(dict(comparison=label, mean_macro_F2_diff=m, ci_low=lo, ci_high=hi,
                              verdict=verdict))
boot = pd.DataFrame(boot_rows)
boot.to_csv(f"{FIG}/bootstrap_differences.csv", index=False)
print(boot.round(4).to_string())
```

```
# %% [6] Figure: headline results
fig, ax = plt.subplots(figsize=(9, 4.5))
x = np.arange(len(NAMES))
ax.bar(x - 0.2, [res[n]["macro_F2"] for n in NAMES], 0.4, label="Macro F2", color="#3FB6A8")
ax.bar(x + 0.2, [res[n]["samples_F2 (Kaggle metric)"] for n in NAMES], 0.4,
       label="Kaggle-style F2 (per-tile average)", color="#7EE081")
ax.axhline(0.85, color="#FF8A3D", ls="--", label="Target 0.85")
ax.set_xticks(x)
ax.set_xticklabels([n.split("_", 1)[0] for n in NAMES])
ax.set_ylim(0, 1)
ax.set_ylabel("F2 on the test set")
ax.legend()
ax.set_title("Test results by experiment")
plt.tight_layout()
plt.savefig(f"{FIG}/fig_results.png", dpi=200)
```

```
plt.close()
```

```
# %% [7] Per-label F2 for the best experiment, rare labels highlighted
best = max(NAMES, key=lambda n: res[n]["macro_F2"])
f2l = f2_per_label(store[best]["y"], store[best]["pred"])
order = np.argsort(f2l)
fig, ax = plt.subplots(figsize=(8, 6))
ax.barh([LABELS[i] for i in order], f2l[order],
        color=["#FF8A3D" if i in RARE_IDX else "#3FB6A8" for i in order])
ax.set_xlabel("F2 on test")
ax.set_title(f"Per-label F2: {best} (orange = rare, high-impact)")
plt.tight_layout()
plt.savefig(f"{FIG}/fig_per_label_f2.png", dpi=200)
plt.close()
pd.DataFrame({"label": LABELS, "F2": f2l}).sort_values("F2").to_csv(
    f"{FIG}/per_label_f2_{best}.csv", index=False)
```

```
# %% [8] Precision/recall curves for the rare labels (RGB vs NIR+NDVI, same architecture)
pair = ("C_resnet_rgb", "D_resnet_nir_ndvi") if "D_resnet_nir_ndvi" in store \
    else (NAMES[0], NAMES[-1])
fig, axes = plt.subplots(1, len(RARE), figsize=(4 * len(RARE), 3.6), squeeze=False)
for ax, l in zip(axes[0], RARE):
    j = LABELS.index(l)
    for n, c in zip(pair, ["#9DB5A6", "#FF8A3D"]):
        p, r, _ = precision_recall_curve(store[n]["y"][:, j], store[n]["probs"][:, j])
        ax.plot(r, p, color=c, label=n.split("_", 1)[0])
    ax.set_title(l)
    ax.set_xlabel("recall")
    ax.set_ylabel("precision")
    ax.set_ylim(0, 1.02)
    ax.legend()
plt.tight_layout()
plt.savefig(f"{FIG}/fig_pr_rare.png", dpi=200)
plt.close()
```

```
# %% [9] Confusion matrices (one 2 x 2 per label) for the best experiment
cms = multilabel_confusion_matrix(store[best]["y"], store[best]["pred"].astype(int))
fig, axes = plt.subplots(3, 6, figsize=(15, 7.5))
for ax in axes.ravel():
    ax.axis("off")
for j, ax in enumerate(axes.ravel()[:17]):
    ax.axis("on")
    ax.imshow(cms[j], cmap="Greens")
    for (a, b), v in np.ndenumerate(cms[j]):
        ax.text(b, a, int(v), ha="center", va="center", fontsize=9)
    ax.set_title(LABELS[j], fontsize=9)
    ax.set_xticks([0, 1])
    ax.set_xticklabels(["pred 0", "pred 1"], fontsize=7)
    ax.set_yticks([0, 1])
    ax.set_yticklabels(["true 0", "true 1"], fontsize=7)
plt.tight_layout()
plt.savefig(f"{FIG}/fig_confusion_matrices.png", dpi=200)
plt.close()
```

```
# %% [10] Headline summary to copy into the deck
print("\nBest experiment by macro F2:", best)
for n in NAMES:
    r = res[n]
    print(f"{n}: macro F2 {r['macro_F2']:.3f} | Kaggle-style F2 "
          f"{r['samples_F2 (Kaggle metric)']:.3f} | rare recall {r['rare_recall_mean']:.3f}")
print("Targets: macro F2 >= 0.85, rare recall >= 0.60. Report honestly if you miss them.")
print("Figures saved in", FIG)
```